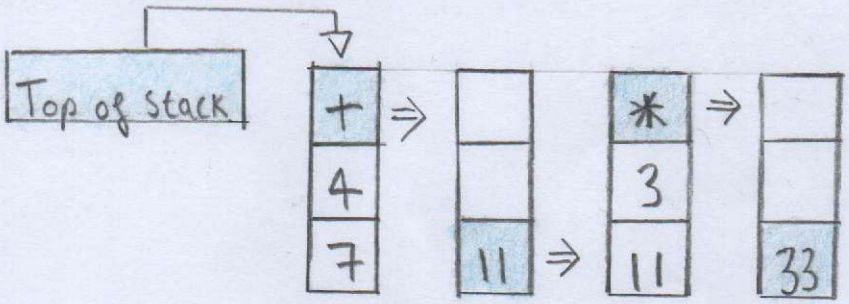
MOTIVATION AND PLANNING OF ONE_STACK

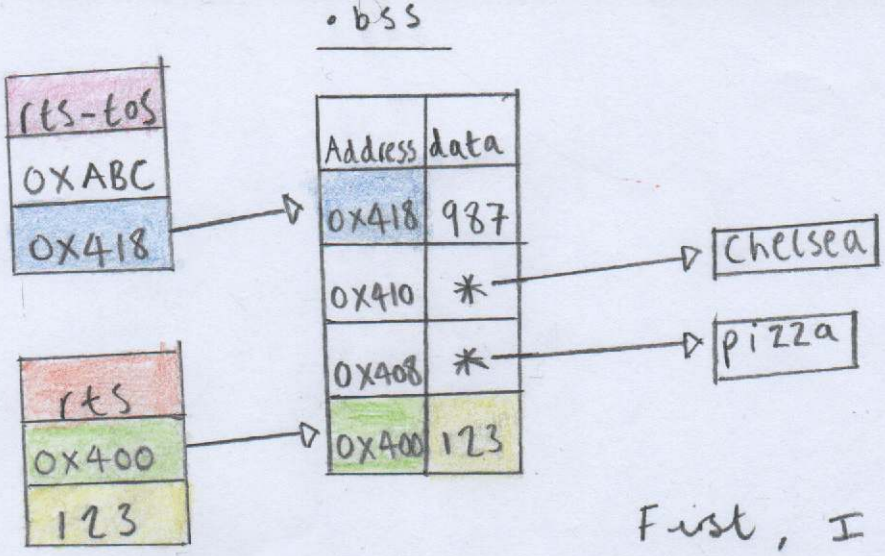
I used 'dc' on my linux pc and thought it was a strange but interesting way of working out maths. The stack 'dc' uses can be thought of a stack used in programming (FILO/LIFO), Like the base pointer being used after we drop down the stack pointer. The more I used 'dc', the more I became interested and thought to myself about copying this and making my own opcords to make a language.



If I entered 7, 4, +, 3, * in 'dc', it would calculate the answer one step at a time. The operands '+' and '*' would look at the values already on the stack.

I knew that different data types have different sizes in bytes, a char is 1 byte and pointers in X86-64 are 8 bytes. To make things easy for me I thought that each space on the stack would be 8 bytes and this would let me store any data type. I called this the run time stack, rts for short.

I needed to mark and keep track of the top of the stack (the same way rsp does), so I made an 8 byte var called rts_tos (runtime stack, top of stack). This would store the address of the top of my runtime stack.



rts was declared in .bss section and I resq x100 bytes for the run time stack, rts.

To push values onto the rts, it's easy.

First, I move rts-tos up 8 bytes, here is the code:

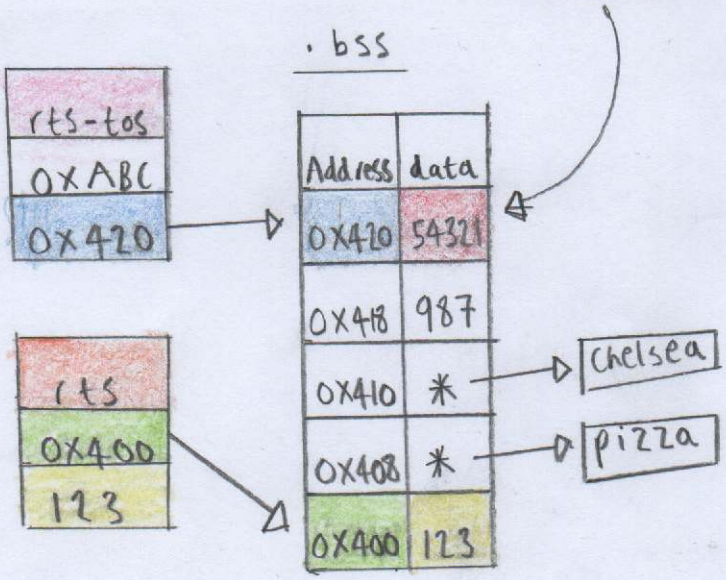
```
add     qword [rts-tos], 0x08
```

or I can use a pointer to do this, I'll use register r8.

```
Mov     r8, rts-tos
add     [r8], 0x08
```

Now that I have made space on the run time stack, I can add data.

```
Mov     r8, [r8]        ; r8 points to address of top of stack
Mov     [r8], 54321     ; r8 updates the value here
```



I can also do the reverse of this and pop off the stack.

```
Sub     qword [rts-tos], 0x08
```

or:

```
Mov     r8, rts-tos
sub     [r8], 0x08
```

The run time stack could be on the heap segment or on the global .data /.bss segment. I thought that I could use the .bss segment rather than the heap segment, as it will be easier to have global scope access. I Know that global variables can be risky, but I decided to initialise the stack in the .bss segment at compile time:

```
section .bss:

        rts         resq:100 ; stack size 100 x8bytes
        rts-tos     resq:1   ; pointer to top of data
```

Now I have a single stack, any data can pushed onto the top. When I pop off, I just make the pointer -8 bytes and leave the value in the stack, because the next time I push on, it will be overwritten. The same way rbp and rsp leave values in their stack segment when functions are called or when they finish.